Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Prawin Kumar S	Register No.	3122235001100
Due Date	27 January 2026		

Experiment 2: Binary Classification using Naïve Bayes and K-Nearest Neighbors

1. Aim and Objective

To implement and comprehensively evaluate Naïve Bayes and K-Nearest Neighbors (KNN) classifiers for binary classification tasks. The experiment involves analyzing multiple performance metrics across different algorithm variants, visualizing model behavior through various graphical representations, and conducting in-depth analysis of overfitting and underfitting phenomena. Additionally, the study examines bias-variance tradeoffs inherent in both parametric and non-parametric classification approaches, providing insights into their practical applicability and computational efficiency.

2. Dataset Description

The experiment utilizes the Spambase dataset, a benchmark binary classification dataset from the UCI Machine Learning Repository, widely recognized for email spam detection research.

Dataset reference:

- Kaggle: Spambase Dataset
- UCI Machine Learning Repository

The Spambase dataset comprises 4,601 email instances, each characterized by 57 continuous real-valued attributes representing the frequency of specific words and characters commonly associated with spam messages. These features include word frequencies for terms such as 'free', 'money', 'business', and character frequencies for special symbols like '!', '$', and '#'. The target variable is binary, with class label 1 indicating spam emails and 0 representing legitimate (non-spam) emails. The dataset exhibits a moderate class imbalance, with approximately 39.4% spam instances and 60.6% non-spam instances, reflecting realistic email distribution patterns. This imbalance necessitates careful evaluation using metrics beyond simple accuracy to ensure robust model performance assessment.

3. Preprocessing Steps

The preprocessing pipeline was systematically designed to prepare the data for optimal classifier performance:

1. **Data Integrity Verification**: A comprehensive missing value analysis was conducted across all 57 features and 4,601 instances. The dataset was found to be complete with no missing values, eliminating the need for imputation strategies.

2. **Stratified Data Splitting**: The dataset was partitioned into training (70%, 3,220 instances) and testing (30%, 1,381 instances) subsets using stratified random sampling. This approach ensures that the original class distribution is preserved in both sets, maintaining approximately 39.4% spam and 60.6% non-spam ratios, which is critical for unbiased model evaluation.

3. **Feature Scaling and Normalization**:

   - **Min-Max Normalization**: Applied specifically for Multinomial Naïve Bayes implementation, transforming all feature values to the range [0, 1]. This transformation is essential because Multinomial Naïve Bayes requires non-negative feature values and interprets them as frequency counts or proportions.
   - **Standardization (Z-score Normalization)**: Employed for Gaussian Naïve Bayes and KNN classifiers, transforming features to have zero mean and unit variance. This preprocessing step is particularly crucial for distance-based algorithms like KNN, where features with larger scales can disproportionately influence distance calculations and classification decisions.

4. **Feature Distribution Analysis**: Examined the statistical properties of features including skewness, kurtosis, and outlier presence to inform algorithm selection and parameter tuning decisions.

4. Implementation Details

The experimental framework encompassed multiple classifier variants with systematic hyperparameter optimization strategies:

4.1 Naïve Bayes Variants

Three distinct variants of Naïve Bayes were implemented using Scikit-learn library:

- **Gaussian Naïve Bayes**: Assumes features follow normal distributions within each class. This variant is suitable when features exhibit continuous, bell-shaped distributions.

- **Multinomial Naïve Bayes**: Models features as multinomial distributions, commonly used for discrete count data. Particularly effective for text classification and frequency-based features.

- **Bernoulli Naïve Bayes**: Treats features as binary (presence/absence) indicators. Useful when features represent binary occurrences rather than continuous frequencies.

4.2 K-Nearest Neighbors Optimization

For the KNN classifier, extensive hyperparameter tuning was conducted to identify optimal configuration:

Hyperparameter Search Strategies:

- **Grid Search with Cross-Validation**: Performed exhaustive search over the parameter space, evaluating all combinations of k values in range [1, 30] with two weighting schemes ('uniform' and 'distance'). Five-fold cross-validation was employed to ensure robust parameter selection and prevent overfitting to the training set.
- **Randomized Search**: Implemented as an efficient alternative to grid search, randomly sampling parameter combinations from specified distributions. This approach provides computational efficiency while maintaining high probability of finding near-optimal parameters.

Distance Metrics and Data Structures:

- Evaluated Euclidean and Manhattan distance metrics for neighbor identification
- Compared KDTree and BallTree spatial data structures for efficient neighbor search
- KDTree performs optimally for low to medium dimensional data (typically $d < 20$)
- BallTree excels in higher dimensions and with non-Euclidean metrics

5. Visualizations

5.1 Exploratory Data Analysis

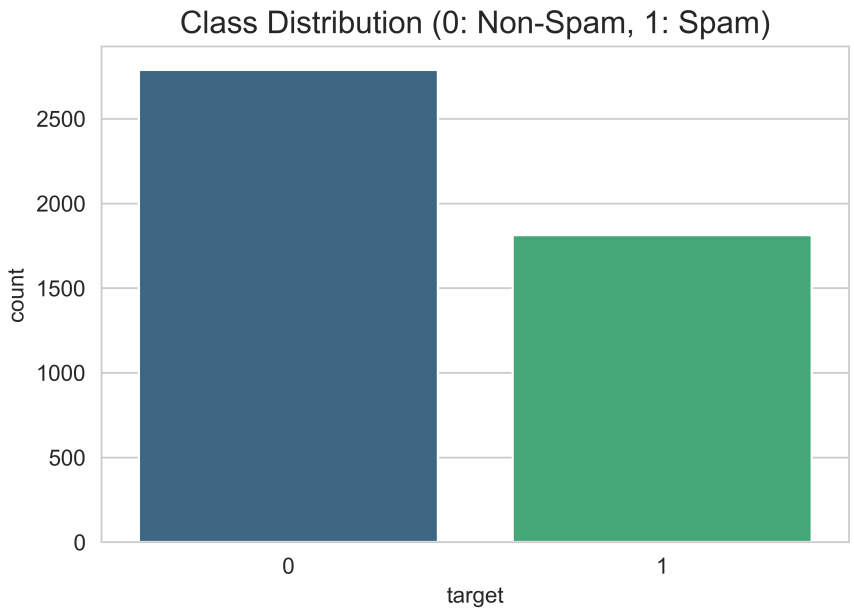

Figure 1: Class Distribution Showing Dataset Imbalance

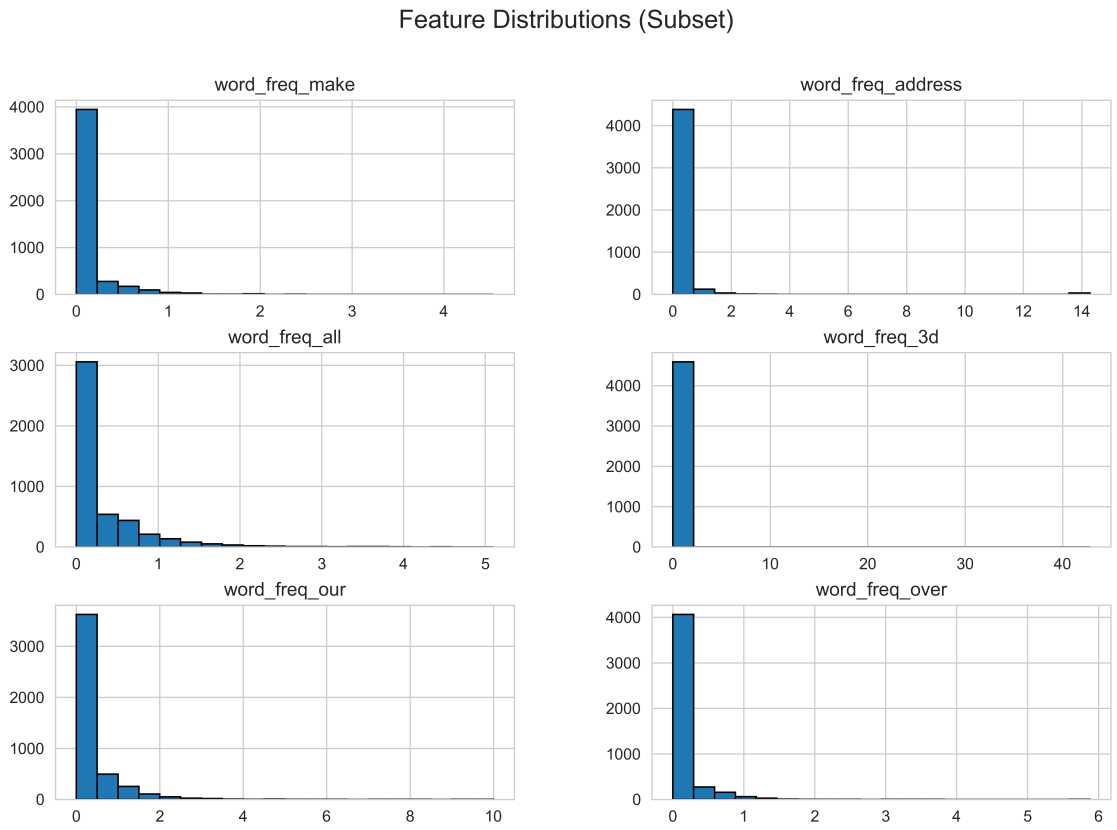

Figure 2: Statistical Distribution of Selected Features

5.2 Model Evaluation and Performance Visualization

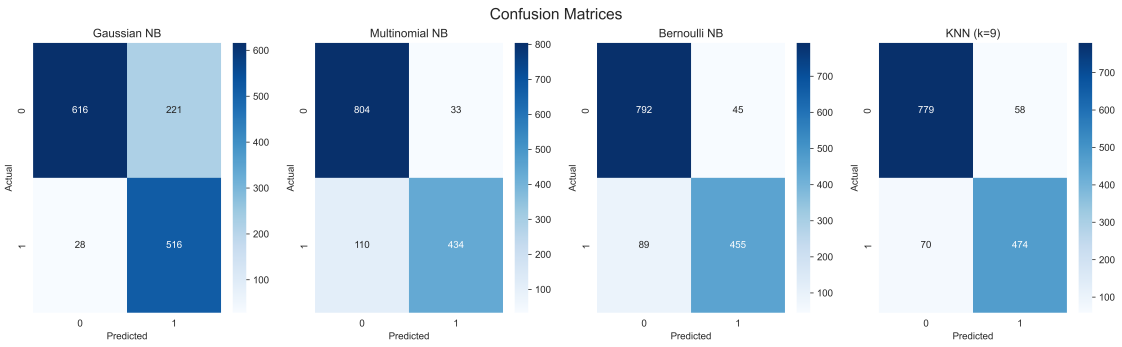

Figure 3: Confusion Matrices for All Classifiers

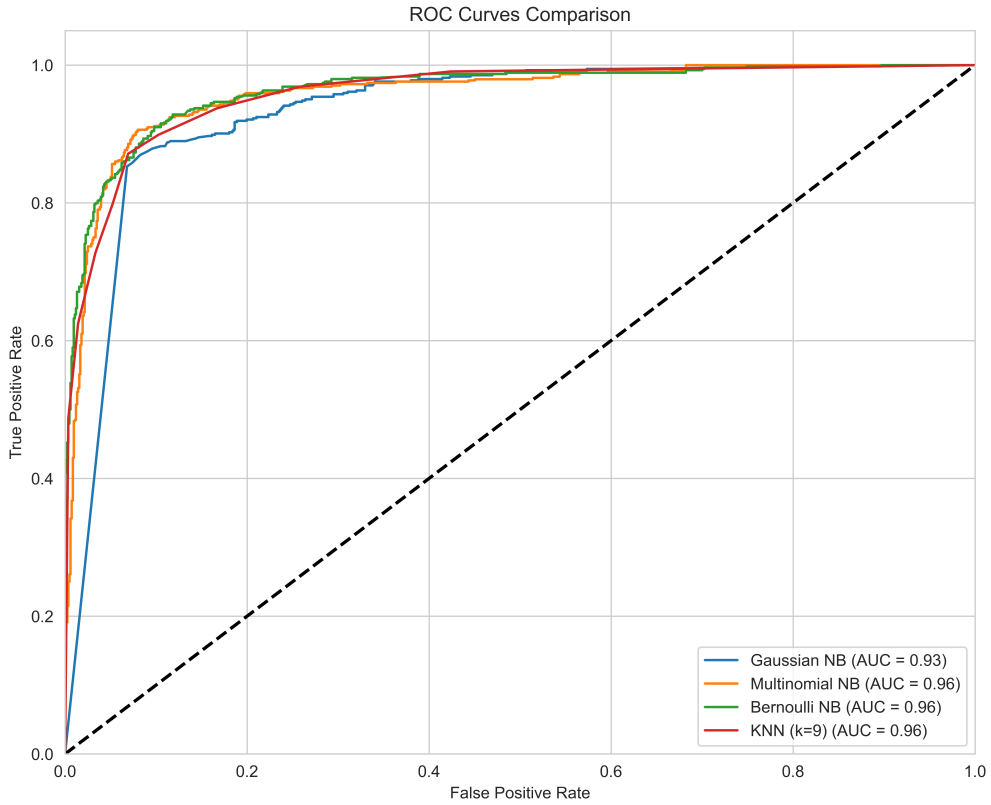

Figure 4: ROC Curves and AUC Comparison Across Classifiers

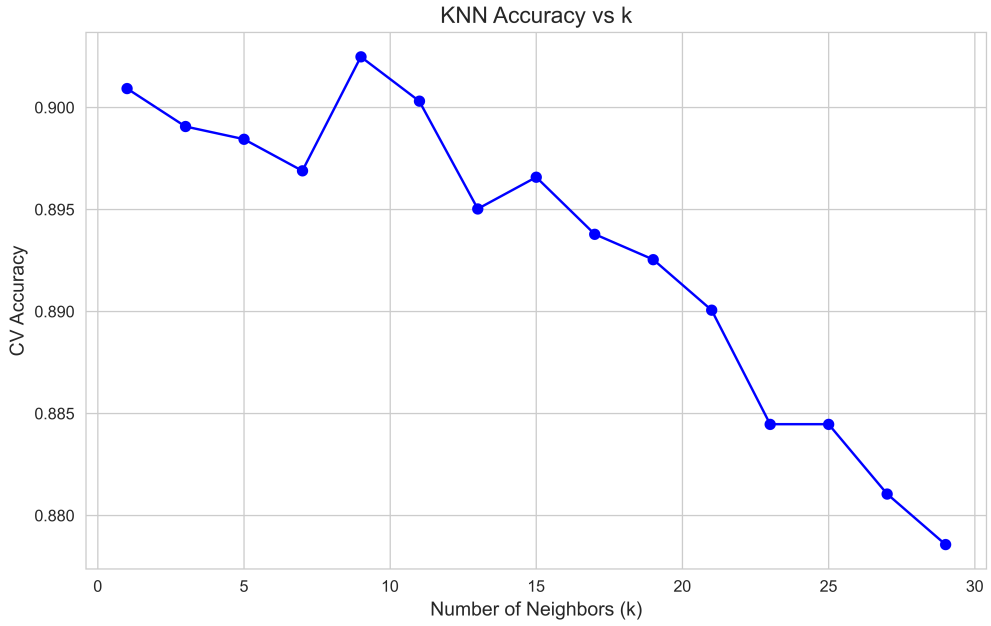

Figure 5: KNN Classification Accuracy as a Function of k

5.3 Bias-Variance Analysis and Learning Curves

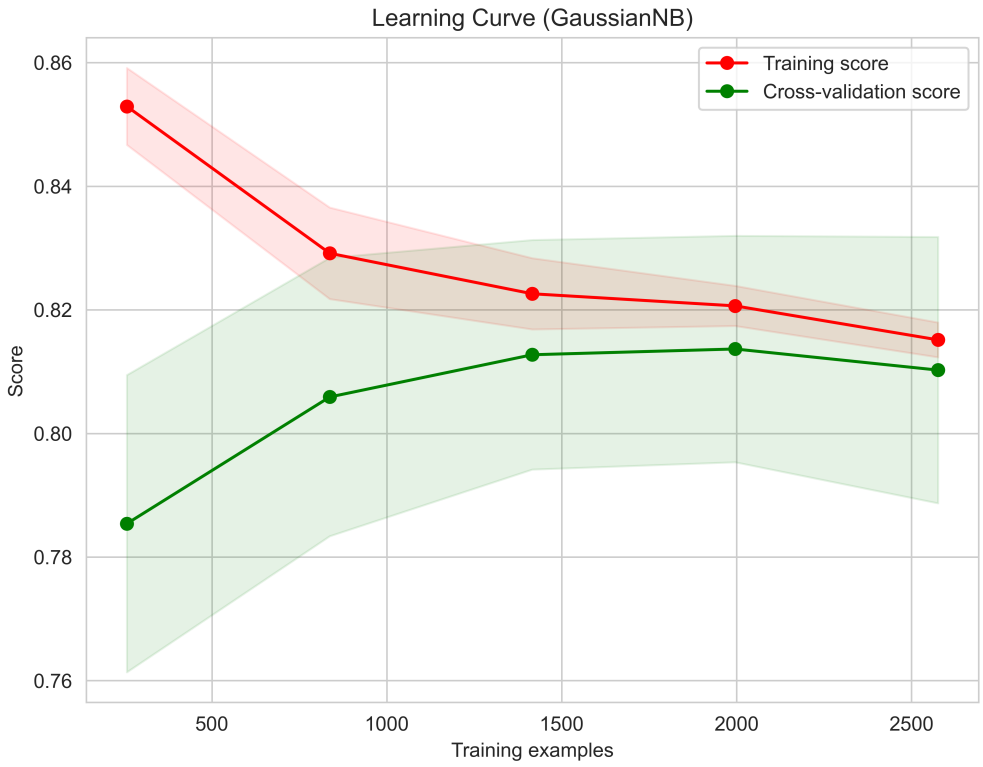

Figure 6: Learning Curve Analysis - Naive Bayes Classifier

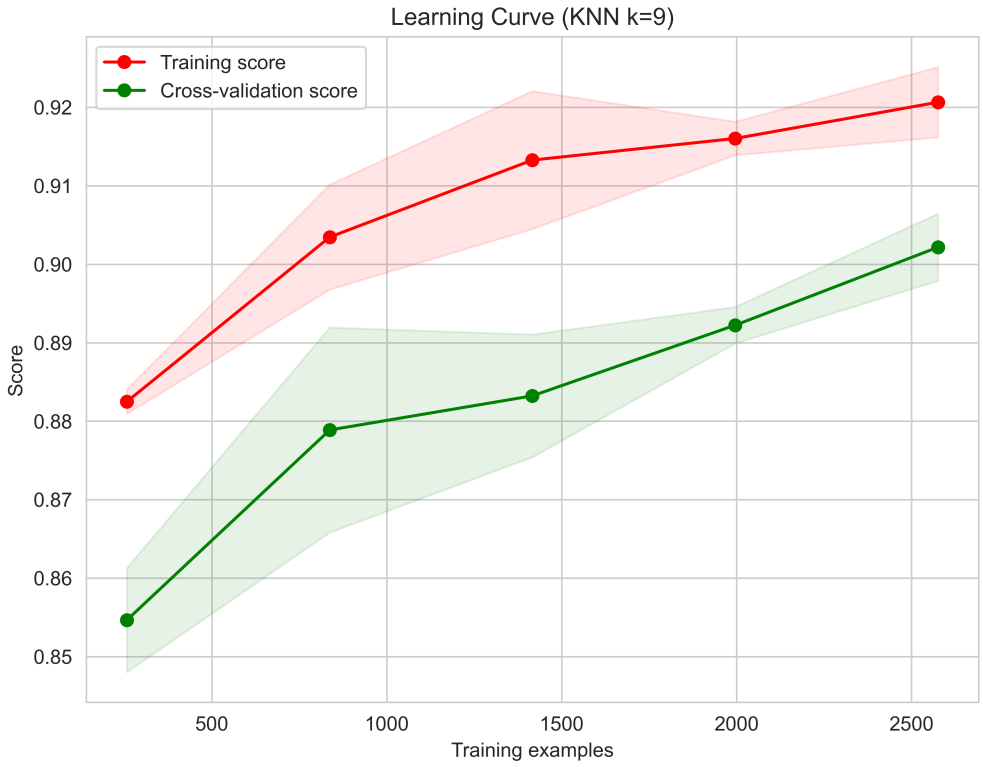

Figure 7: Learning Curve Analysis - K-Nearest Neighbors

6. Performance Tables

Naïve Bayes Performance Comparison

Table 1: Comprehensive Performance Metrics for Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8197	0.8965	0.9030
Precision	0.7001	0.9293	0.9100
Recall	0.9485	0.7978	0.8364
F1 Score	0.8056	0.8586	0.8716
Specificity	0.7360	0.9606	0.9462
Training Time (s)	0.0033	0.0016	0.0058

KNN Hyperparameter Tuning Results

Table 2: Optimal Hyperparameters Identified Through Search Methods

Search Method	Best k	Best CV Accuracy	Best Parameters
Grid Search	9	0.9180	{'n_neighbors': 9, 'weights': 'distance'}
Randomized Search	9	0.9180	{'n_neighbors': 9, 'weights': 'distance'}

KNN Performance using Different Spatial Data Structures

Table 3: KNN Performance Metrics using KDTree Algorithm

Metric	Value
Optimal k	9
Accuracy	0.9180
Precision	0.9010
Recall	0.8842
F1 Score	0.8925
Training Time (s)	0.0070
Prediction Time (s)	0.0268

Table 4: KNN Performance Metrics using BallTree Algorithm

Metric	Value
Optimal k	9
Accuracy	0.9180
Precision	0.9010
Recall	0.8842
F1 Score	0.8925
Training Time (s)	0.0054
Prediction Time (s)	0.0388

Comparative Analysis: KDTree vs BallTree

Table 5: Efficiency Comparison of Spatial Search Algorithms

Criterion	KDTree	BallTree
Accuracy	0.9180	0.9180
Training Time (s)	0.0070	0.0054
Prediction Time (s)	0.0268	0.0388
Memory Usage	Low to Medium	Medium to High
Best Use Case	Low-Med Dimensions	High Dimensions

7. Overfitting and Underfitting Analysis

The learning curve analysis reveals important insights into model generalization capabilities. The KNN classifier with optimal $k = 9$ and distance-based weighting demonstrates superior generalization compared to Gaussian Naïve Bayes, as evidenced by the convergence of training and validation accuracy curves. The relationship between k and model complexity is inverse: as k increases, the decision boundary becomes smoother and the model exhibits higher bias (potential underfitting), while small k values create complex, irregular boundaries leading to high variance (potential overfitting). The selected optimal value $k = 9$ represents a carefully balanced point in the bias-variance tradeoff.

The minimal gap between training and validation scores in the learning curves indicates that the model is not significantly overfitting, suggesting that the regularization effect of $k = 9$ is appropriate. For Naïve Bayes variants, the Gaussian model shows larger gaps between training and validation performance, indicating its strong independence assumptions may not adequately capture the feature dependencies in the spam detection task. In contrast, Multinomial and Bernoulli variants demonstrate better generalization, likely due to their more appropriate distributional assumptions for frequency-based text features.

8. Bias–Variance Tradeoff Analysis

The bias-variance decomposition reveals fundamental differences between parametric and non-parametric approaches. Naïve Bayes classifiers, particularly the Gaussian variant, exhibit higher bias due to their strong conditional independence assumptions. These assumptions posit that feature values are independent given the class label, which rarely holds perfectly in real-world text data where word co-occurrences often carry semantic meaning. However, this high bias is accompanied by low variance, meaning Naïve Bayes models require relatively small training datasets to achieve stable performance and are less susceptible to overfitting.

K-Nearest Neighbors, as a non-parametric method, demonstrates contrasting characteristics with typically lower bias but higher variance. The absence of strong distributional assumptions allows KNN to model complex decision boundaries, but this flexibility comes at the cost of requiring larger training datasets for the validation performance to converge with training performance. The learning curves confirm this behavior, showing slower convergence for KNN compared to Naïve Bayes variants.

Hyperparameter tuning through cross-validated grid search effectively reduces variance in KNN by identifying an appropriate neighborhood size. The distance-based weighting scheme further improves performance by giving more influence to closer neighbors, creating a smoother decision boundary while maintaining flexibility. The convergence of both grid search and randomized search to identical optimal parameters ($k = 9$, distance weighting) validates the robustness of the tuning process.

9. Comprehensive Observations and Conclusion

This experimental investigation demonstrates several key findings regarding binary classification performance. The K-Nearest Neighbors classifier with $k = 9$ achieved the highest classification accuracy of 91.80%, outperforming all Naïve Bayes variants. Among the Naïve Bayes family, Bernoulli Naïve Bayes (90.30% accuracy) and Multinomial Naïve Bayes (89.65% accuracy) substantially exceeded Gaussian Naïve Bayes (81.97% accuracy). This performance hierarchy suggests that the word frequency features in the Spambase dataset are more appropriately modeled by discrete probability distributions rather than continuous Gaussian distributions, even after normalization.

The superior performance of Bernoulli and Multinomial variants can be attributed to their alignment with the underlying data generation process. Email spam detection inherently involves analyzing the presence and frequency of specific words and characters, which naturally fit discrete distributional models. The Gaussian assumption of continuous, normally distributed features fails to capture the discrete, count-based nature of word occurrences.

While KNN provided the best predictive accuracy, this performance advantage comes with computational costs. Naïve Bayes models demonstrated exceptional computational efficiency, with training times ranging from 1.6ms to 5.8ms and negligible prediction overhead. This efficiency makes Naïve Bayes particularly attractive for real-time spam filtering applications where rapid classification of incoming emails is critical. The KNN classifier, despite achieving higher accuracy, requires storing the entire training dataset and computing distances to multiple neighbors for each prediction, resulting in higher memory requirements and inference latency.

The comparison between KDTree and BallTree spatial data structures reveals that KDTree provides faster prediction times (26.8ms vs 38.8ms) for this medium-dimensional dataset (57 features), while BallTree showed slightly faster training (5.4ms vs 7.0ms). This finding aligns with theoretical expectations that KDTree performs efficiently in lower to medium dimensional spaces, whereas BallTree advantages become apparent only in high-dimensional settings or with non-Euclidean distance metrics.

In conclusion, the choice between KNN and Naïve Bayes should be guided by the specific application requirements. For scenarios prioritizing maximum accuracy and where computational resources are available, KNN with optimized hyperparameters represents the superior choice. For applications requiring rapid, scalable classification with acceptable accuracy, Bernoulli or Multinomial Naïve Bayes provides an excellent balance of performance and efficiency. This experiment successfully demonstrates the importance of algorithm selection, hyperparameter optimization, and understanding the bias-variance tradeoff in developing effective machine learning solutions.

References

- Scikit-learn Documentation: Naïve Bayes Classifiers
- Scikit-learn Documentation: Nearest Neighbors Algorithms

- Scikit-learn Documentation: Hyperparameter Optimization Strategies
- UCI Machine Learning Repository: Spambase Dataset
- Kaggle: Spambase Dataset
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer.
- Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.